AI Trace Observatory

Project overview

Inspect synthetic AI request traces, calculate service metrics, and detect threshold breaches without an external telemetry service.

This guide explains the delivered implementation, how to run it, what each component does, the algorithms, and the limits of the demonstration.

Prepared for AI Product Reviewer / Beta-Testing internship interviews. Synthetic data. Local demo. AI-assisted starter; personalize and understand before presenting.

AI Trace Observatory

Problem and demo

AI features need operational visibility beyond whether one response looks good.

This dashboard reads four synthetic request traces and calculates request count, error rate, tail latency, token use and illustrative cost.

Run with model all, then demo-small.

Lower the latency threshold to create more alerts.

Explain that the dashboard replays fixture data and does not instrument a live model.

AI Trace Observatory

Setup and repository map

Prerequisite: Python 3.10 or newer.

No pip install or API key is required.

Extract this ZIP, open a terminal in its project folder and run python server.py.

Open http://127.0.0.1:8000 in your browser.

If that port is busy, run python server.py --port 8001 and open port 8001 instead.

On Windows, py can replace python.

Stop with Ctrl+C.

Run python -m unittest discover -v to execute tests.

logic.py holds domain logic; server.py handles HTTP and local storage; index.html contains the browser UI; sample_data.json holds synthetic fixtures; test_logic.py checks domain behavior; test_http.py checks the HTTP boundary.

requirements.txt documents the standard-library-only runtime.

The guide-generation tools are not runtime dependencies.

AI Trace Observatory

Architecture and technology choices

Browser form -> fetch POST /api/run -> Python HTTP handler -> domain run(payload, records) -> JSON result -> safe DOM tables.

GET / serves the app, GET /api/health returns a health response, and unknown paths return 404.

Python provides a portable runtime; http.server provides a minimal HTTP boundary; JSON keeps fixtures inspectable; HTML and CSS provide a responsive UI; JavaScript fetch connects the UI and backend; unittest provides repeatable checks.

Browser output uses textContent rather than HTML injection.

There is no framework, build system, external CDN, telemetry upload or database dependency.

Metrics explained

Error rate is non-ok requests divided by all matching requests.

P95 uses nearest rank: sort latencies and select ceiling(0.95 * N), with one-based indexing.

With four samples, p95 is the maximum, so this dataset cannot support reliable service-level conclusions.

Total tokens sum input and output tokens.

Cost is the sum of input tokens times input rate plus output tokens times output rate, divided by one million.

Rates are invented demo values stored per trace.

AI Trace Observatory

Code walkthrough

logic.py filters records by model, validates a finite positive threshold, sorts latencies, aggregates token and cost data and generates an alert for each error or slow request.

A trace that is both slow and failed is represented once with request error as its reason.

server.py exposes the calculation through POST /api/run.

The UI renders scalar metrics as cards and traces and alerts as tables.

Empty filters return zero counts and null p95.

Instrumentation and privacy

For live use, wrap provider calls in a timer, generate a correlation ID, record status, provider-reported usage, model version and prompt-template version.

Keep sensitive prompt content out of default logs.

Separate input and output prices, store the effective date, and compare billed usage against estimates.

Failed requests may still incur charges; the sample deliberately includes input token cost for a timeout.

Real provider billing semantics vary.

Interview discussion and next steps

Describe why averages hide slow experiences and why p95 needs enough samples.

Distinguish an operational error from a low-quality answer: an HTTP-successful response can still hallucinate.

Link trace IDs to evaluation results to connect product quality and reliability.

Extend the dashboard with time windows, histograms, a real collector and alert deduplication.

Use an established telemetry format and avoid high-cardinality sensitive labels.

API, errors and operational limits

POST /api/run accepts a JSON object with Content-Type application/json.

The form fields below correspond to payload keys.

Successful calls return HTTP 200 and JSON; validation or malformed JSON returns 400; unknown paths return 404.

Bodies must be between 1 and 32768 bytes.

Unexpected local data or storage errors return 500 with a generic message.

The server binds to 127.0.0.1 and is intended for a trusted local demo.

Python http.server is not a production server.

Authentication, TLS, concurrent workers, rate limits and schema validation must be designed before deployment.

Do not expose this process directly to the internet.

Example request payload

```
{
  "model": "all",
  "latency_threshold_ms": "1500"
}
```

Testing, GitHub and ownership

Run the logic and HTTP tests before changing sample data, since expected values describe the shipped fixture.

The HTTP tests start a temporary local server and verify health, successful workflow, malformed JSON, non-object payloads and unknown paths.

This is a useful smoke suite, not comprehensive security or browser coverage.

Create a GitHub repository, copy the extracted project files into it, then run git init, git add ., git commit -m "Initial portfolio demo", git branch -M main, git remote add origin YOUR_REPOSITORY_URL, and git push -u origin main.

Replace the URL with your own empty repository.

Add a screenshot and a short video after running it.

This is an AI-assisted starter project; describe your own review, changes and learning accurately.

Do not claim customer usage, production deployment or measured model improvements from these synthetic examples.